NOUS AI ACADEMY / INTENSIVE FILE

TECHNICAL LEARNING FILE

DEEP-04

SQL for Data and AI

Queries, feature tables, and data quality

FORMAT	LENGTH	ACCESS
INTENSIVE FILE	50 PAGES	OFFLINE

Edition 2 - original curriculum - editorial review recommended

FILE / ORIENTATION

Purpose

01 FILE GUIDANCE

This optional advanced guide does not gate the core learning path. Apply sql for data and ai with explicit assumptions, tests, tradeoffs, and limitations.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Prerequisites

01 FILE GUIDANCE

Complete the related core track or pass its diagnostic.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Study Method

01 FILE GUIDANCE

For each unit, explain the concept, follow the workflow, reproduce the example, analyze a failure, and complete the applied lab. Record evidence locally.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

UNIT 01 / CONCEPT

Relational Modeling: Concept

01 OBJECTIVE

Explain and apply relational modeling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Relational Modeling is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In SQL for Data and AI, the terms Relational, Modeling are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should relational modeling be used, and what observable evidence would make that choice defensible? Build a small local example that applies relational modeling, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Relational Modeling in one precise sentence and name one assumption.
2. Create a two-step example of relational modeling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKFLOW

Relational Modeling: Workflow

01 OBJECTIVE

Explain and apply relational modeling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Relational Modeling is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to relational modeling.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies relational modeling, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Relational Modeling in one precise sentence and name one assumption.
2. Create a two-step example of relational modeling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKED EXAMPLE

Relational Modeling: Worked Example

01 OBJECTIVE

Explain and apply relational modeling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies relational modeling, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns relational modeling into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Relational Modeling in one precise sentence and name one assumption.
2. Create a two-step example of relational modeling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / FAILURE ANALYSIS

Relational Modeling: Failure Analysis

01 OBJECTIVE

Explain and apply relational modeling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how relational modeling can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to relational modeling. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Relational Modeling in one precise sentence and name one assumption.
2. Create a two-step example of relational modeling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / APPLIED LAB

Relational Modeling: Applied Lab

01 OBJECTIVE

Explain and apply relational modeling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of relational modeling into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies relational modeling, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Relational Modeling in one precise sentence and name one assumption.
2. Create a two-step example of relational modeling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / CONCEPT

Filtering and Projection: Concept

01 OBJECTIVE

Explain and apply filtering and projection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Filtering and Projection is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In SQL for Data and AI, the terms Filtering, Projection are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should filtering and projection be used, and what observable evidence would make that choice defensible? Build a small local example that applies filtering and projection, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Filtering and Projection in one precise sentence and name one assumption.
2. Create a two-step example of filtering and projection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKFLOW

Filtering and Projection: Workflow

01 OBJECTIVE

Explain and apply filtering and projection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Filtering and Projection is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to filtering and projection.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies filtering and projection, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Filtering and Projection in one precise sentence and name one assumption.
2. Create a two-step example of filtering and projection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKED EXAMPLE

Filtering and Projection: Worked Example

01 OBJECTIVE

Explain and apply filtering and projection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies filtering and projection, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns filtering and projection into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Filtering and Projection in one precise sentence and name one assumption.
2. Create a two-step example of filtering and projection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / FAILURE ANALYSIS

Filtering and Projection: Failure Analysis

01 OBJECTIVE

Explain and apply filtering and projection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how filtering and projection can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to filtering and projection. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Filtering and Projection in one precise sentence and name one assumption.
2. Create a two-step example of filtering and projection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / APPLIED LAB

Filtering and Projection: Applied Lab

01 OBJECTIVE

Explain and apply filtering and projection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of filtering and projection into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies filtering and projection, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Filtering and Projection in one precise sentence and name one assumption.
2. Create a two-step example of filtering and projection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / CONCEPT

Joins: Concept

01 OBJECTIVE

Explain and apply joins using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Joins is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In SQL for Data and AI, the terms Joins are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should joins be used, and what observable evidence would make that choice defensible? Build a small local example that applies joins, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Joins in one precise sentence and name one assumption.
2. Create a two-step example of joins and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKFLOW

Joins: Workflow

01 OBJECTIVE

Explain and apply joins using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Joins is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to joins.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies joins, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Joins in one precise sentence and name one assumption.
2. Create a two-step example of joins and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKED EXAMPLE

Joins: Worked Example

01 OBJECTIVE

Explain and apply joins using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies joins, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns joins into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Joins in one precise sentence and name one assumption.
2. Create a two-step example of joins and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / FAILURE ANALYSIS

Joins: Failure Analysis

01 OBJECTIVE

Explain and apply joins using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how joins can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to joins. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Joins in one precise sentence and name one assumption.
2. Create a two-step example of joins and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / APPLIED LAB

Joins: Applied Lab

01 OBJECTIVE

Explain and apply joins using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of joins into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies joins, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Joins in one precise sentence and name one assumption.
2. Create a two-step example of joins and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / CONCEPT

Aggregation: Concept

01 OBJECTIVE

Explain and apply aggregation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Aggregation is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In SQL for Data and AI, the terms Aggregation are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should aggregation be used, and what observable evidence would make that choice defensible? Build a small local example that applies aggregation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Aggregation in one precise sentence and name one assumption.
2. Create a two-step example of aggregation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKFLOW

Aggregation: Workflow

01 OBJECTIVE

Explain and apply aggregation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Aggregation is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to aggregation.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies aggregation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Aggregation in one precise sentence and name one assumption.
2. Create a two-step example of aggregation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKED EXAMPLE

Aggregation: Worked Example

01 OBJECTIVE

Explain and apply aggregation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies aggregation, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns aggregation into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Aggregation in one precise sentence and name one assumption.
2. Create a two-step example of aggregation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / FAILURE ANALYSIS

Aggregation: Failure Analysis

01 OBJECTIVE

Explain and apply aggregation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how aggregation can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to aggregation. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Aggregation in one precise sentence and name one assumption.
2. Create a two-step example of aggregation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / APPLIED LAB

Aggregation: Applied Lab

01 OBJECTIVE

Explain and apply aggregation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of aggregation into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies aggregation, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Aggregation in one precise sentence and name one assumption.
2. Create a two-step example of aggregation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / CONCEPT

Window Functions: Concept

01 OBJECTIVE

Explain and apply window functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Window Functions is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In SQL for Data and AI, the terms Window, Functions are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should window functions be used, and what observable evidence would make that choice defensible? Build a small local example that applies window functions, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Window Functions in one precise sentence and name one assumption.
2. Create a two-step example of window functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKFLOW

Window Functions: Workflow

01 OBJECTIVE

Explain and apply window functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Window Functions is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to window functions.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies window functions, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Window Functions in one precise sentence and name one assumption.
2. Create a two-step example of window functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKED EXAMPLE

Window Functions: Worked Example

01 OBJECTIVE

Explain and apply window functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies window functions, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns window functions into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Window Functions in one precise sentence and name one assumption.
2. Create a two-step example of window functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / FAILURE ANALYSIS

Window Functions: Failure Analysis

01 OBJECTIVE

Explain and apply window functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how window functions can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to window functions. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Window Functions in one precise sentence and name one assumption.
2. Create a two-step example of window functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / APPLIED LAB

Window Functions: Applied Lab

01 OBJECTIVE

Explain and apply window functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of window functions into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies window functions, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Window Functions in one precise sentence and name one assumption.
2. Create a two-step example of window functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / CONCEPT

CTEs: Concept

01 OBJECTIVE

Explain and apply ctes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

CTEs is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In SQL for Data and AI, the terms CTEs are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should ctes be used, and what observable evidence would make that choice defensible? Build a small local example that applies ctes, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define CTEs in one precise sentence and name one assumption.
2. Create a two-step example of ctes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKFLOW

CTEs: Workflow

01 OBJECTIVE

Explain and apply ctes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

CTEs is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to ctes.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies ctes, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define CTEs in one precise sentence and name one assumption.
2. Create a two-step example of ctes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKED EXAMPLE

CTEs: Worked Example

01 OBJECTIVE

Explain and apply ctes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies ctes, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns ctes into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define CTEs in one precise sentence and name one assumption.
2. Create a two-step example of ctes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / FAILURE ANALYSIS

CTEs: Failure Analysis

01 OBJECTIVE

Explain and apply ctes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how ctes can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to ctes. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define CTEs in one precise sentence and name one assumption.
2. Create a two-step example of ctes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / APPLIED LAB

CTEs: Applied Lab

01 OBJECTIVE

Explain and apply ctes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of ctes into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies ctes, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define CTEs in one precise sentence and name one assumption.
2. Create a two-step example of ctes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / CONCEPT

Query Plans: Concept

01 OBJECTIVE

Explain and apply query plans using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Query Plans is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In SQL for Data and AI, the terms Query, Plans are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should query plans be used, and what observable evidence would make that choice defensible? Build a small local example that applies query plans, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Query Plans in one precise sentence and name one assumption.
2. Create a two-step example of query plans and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKFLOW

Query Plans: Workflow

01 OBJECTIVE

Explain and apply query plans using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Query Plans is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to query plans.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies query plans, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Query Plans in one precise sentence and name one assumption.
2. Create a two-step example of query plans and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKED EXAMPLE

Query Plans: Worked Example

01 OBJECTIVE

Explain and apply query plans using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies query plans, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns query plans into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Query Plans in one precise sentence and name one assumption.
2. Create a two-step example of query plans and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / FAILURE ANALYSIS

Query Plans: Failure Analysis

01 OBJECTIVE

Explain and apply query plans using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how query plans can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to query plans. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Query Plans in one precise sentence and name one assumption.
2. Create a two-step example of query plans and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / APPLIED LAB

Query Plans: Applied Lab

01 OBJECTIVE

Explain and apply query plans using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of query plans into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies query plans, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Query Plans in one precise sentence and name one assumption.
2. Create a two-step example of query plans and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / CONCEPT

Feature Tables: Concept

01 OBJECTIVE

Explain and apply feature tables using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Feature Tables is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In SQL for Data and AI, the terms Feature, Tables are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should feature tables be used, and what observable evidence would make that choice defensible? Build a small local example that applies feature tables, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Feature Tables in one precise sentence and name one assumption.
2. Create a two-step example of feature tables and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKFLOW

Feature Tables: Workflow

01 OBJECTIVE

Explain and apply feature tables using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Feature Tables is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to feature tables.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies feature tables, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Feature Tables in one precise sentence and name one assumption.
2. Create a two-step example of feature tables and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKED EXAMPLE

Feature Tables: Worked Example

01 OBJECTIVE

Explain and apply feature tables using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies feature tables, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns feature tables into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Feature Tables in one precise sentence and name one assumption.
2. Create a two-step example of feature tables and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / FAILURE ANALYSIS

Feature Tables: Failure Analysis

01 OBJECTIVE

Explain and apply feature tables using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how feature tables can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to feature tables. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Feature Tables in one precise sentence and name one assumption.
2. Create a two-step example of feature tables and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / APPLIED LAB

Feature Tables: Applied Lab

01 OBJECTIVE

Explain and apply feature tables using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of feature tables into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies feature tables, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Feature Tables in one precise sentence and name one assumption.
2. Create a two-step example of feature tables and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / CONCEPT

Leakage Prevention: Concept

01 OBJECTIVE

Explain and apply leakage prevention using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Leakage Prevention is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In SQL for Data and AI, the terms Leakage, Prevention are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should leakage prevention be used, and what observable evidence would make that choice defensible? Build a small local example that applies leakage prevention, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Leakage Prevention in one precise sentence and name one assumption.
2. Create a two-step example of leakage prevention and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKFLOW

Leakage Prevention: Workflow

01 OBJECTIVE

Explain and apply leakage prevention using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Leakage Prevention is a central part of sql for data and ai because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to leakage prevention.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies leakage prevention, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Leakage Prevention in one precise sentence and name one assumption.
2. Create a two-step example of leakage prevention and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKED EXAMPLE

Leakage Prevention: Worked Example

01 OBJECTIVE

Explain and apply leakage prevention using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies leakage prevention, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns leakage prevention into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Leakage Prevention in one precise sentence and name one assumption.
2. Create a two-step example of leakage prevention and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / FAILURE ANALYSIS

Leakage Prevention: Failure Analysis

01 OBJECTIVE

Explain and apply leakage prevention using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how leakage prevention can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to leakage prevention. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Leakage Prevention in one precise sentence and name one assumption.
2. Create a two-step example of leakage prevention and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / APPLIED LAB

Leakage Prevention: Applied Lab

01 OBJECTIVE

Explain and apply leakage prevention using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of leakage prevention into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies leakage prevention, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations
```

05 PRACTICE

1. Define Leakage Prevention in one precise sentence and name one assumption.
2. Create a two-step example of leakage prevention and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

FILE / ORIENTATION

Deep-Dive Capstone

01 FILE GUIDANCE

Build a compact, reproducible artifact demonstrating sql for data and ai. Include assumptions, a baseline, tests, failure analysis, results, limitations, and instructions for repeating the work offline.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.